

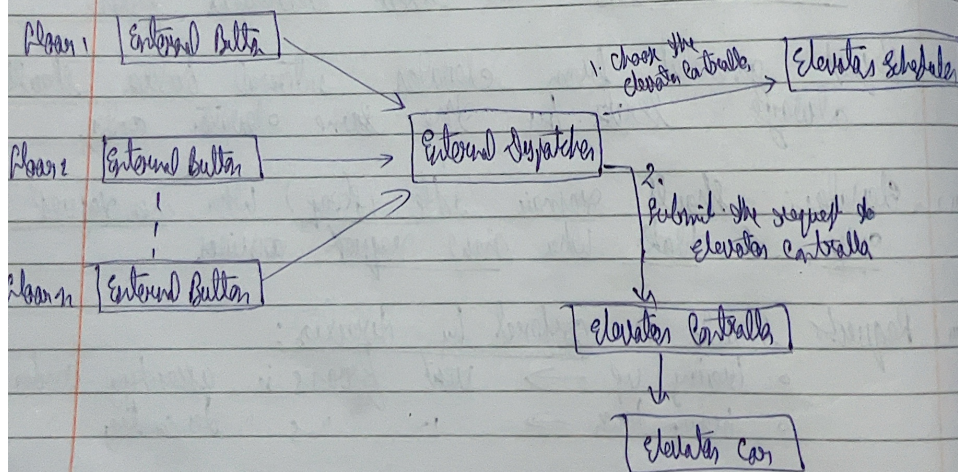
1. Elevator Design UO: Elevator System

Functionality Requirements

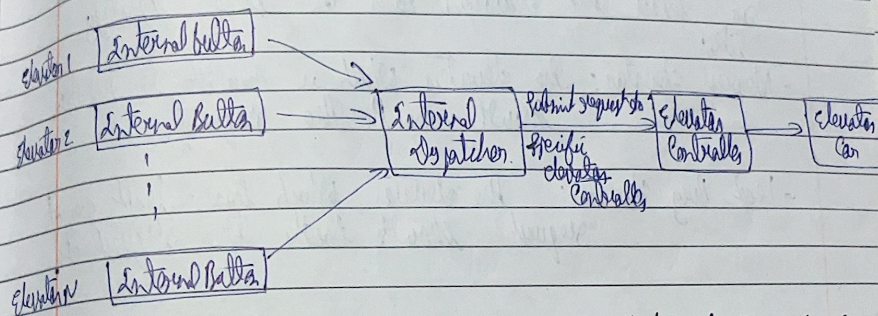
- i. A building has multiple elevators & multiple floors
- ii. A user can request an elevator externally using UP / DOWN buttons at each floor.
 - location
- iii. These UP / DOWN buttons are used to choose the best elevator to serve the request.
- iv. Once elevator is chosen, the floor is added in that particular elev. elevator bucket list
- v. A user inside an elevator can also press an internal button to select destination floor.
- vi. Request generated from elevator internal button should always serve by the same elevator only.
- vii. Elevators should remain idle (sleep) when no request exist & wake when new request arrives.
- viii. Requests should be ordered by direction:
 - being up → visit floors in ascending order
 - being down → " " " descending "

→ Identify objects:

- Building
- floor
- Elevator - Just a POJO
- Elevator Controller - Each elevator is controlled by its controller, which manages its requests.
- External button - Each floor has the External Button
- Internal " - " elevator car has 1 internal button
- Elevator schedule - Maintains the list of elevator controllers, also has elevator block strategy to choose the best elevator to serve the request
- External dispatcher - bridge b/w external buttons & elevator controllers



Internal Dispatcher - Internal buttons can directly call their elevator controller, but sometimes we need to put logging and validation, so it's better to have one internal. Also it follows the same path as external, so it's clear too.



→ Before moving ahead with UML, let's first understand:

1. Elevator Selection Strategy?

2. Elevator moving behaviour (i.e. request should be ordered by direction) What does that mean?

- moving up ⇒ visit floors in ascending order
- moving down ⇒ " " " des " "

- Elevator Selection Strategy:
- When external button is pressed, info which the user is:
 - Floor: Floor at which user is waiting for the lift
 - Direction: Whether user wants to go up or down.

Now when there are more than 1 elevator, then we need to select the elevator based on some strategy

- like:
- Nearest elevator: The elevator which is nearest to the user, should handle the request.
 - Least Busy elevator: The elevator which has least no. of request, should handle the request etc.

- So, selection strategy is:
- Floor info from where the request has come.
 - Direction info, where the user wants to go
 - Elevator info like
 - Is request booked
 - elevator direction
 - " current floor.

& based on the algo, it picks one elevator to handle the request

→ Elevators can moving behaviour (i.e. requests should be ordered by direction):

SCAN algo

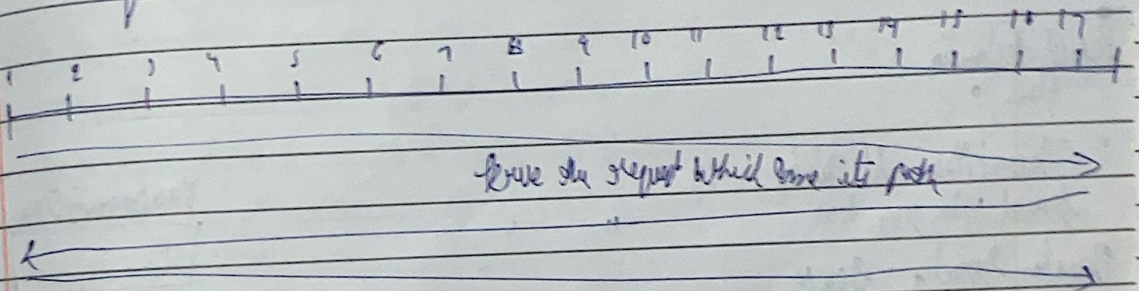
LOOK Algo

SCAN Algo:

Head moves in one direction first, servicing the requests along the way.

Once it reaches till end, it reverses the direction. & it moves End to End

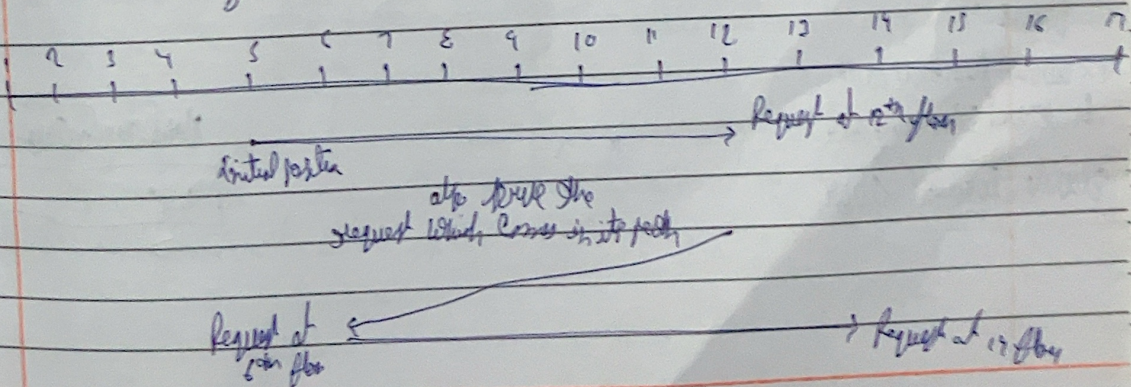
Ex: Elevator going till top floor even if there is no request.



LOOK Algo:

Same as SCAN, but with one change:

- Don't go till end, stop when there is no request



→ We can implement this Look Alike using 2 priority Queue.

1. Min Priority Queue:
Used when elevator is going up, as it need to serve the request in ascending order.
2. Max Priority Queue:
Used when elevator is going down, as it need to serve the request in descending order.

Example:

Elevator direction: UP
Current floor: 5

Min Priority Queue

Max Priority Queue

Request destination floor: 2

2 < Current floor = 5 will be served when going down.
Elevator direction: DOWN

Min Priority Queue
2

Request destination floor: 4

4 < Current floor = 5 will be served when going down.
Elevator direction: DOWN

Min Priority Queue
4, 2
→

let's
Request destination
elevator current
floor = 5
Request when
elevator direction
is up then
not empty

Request destination
elevator current
floor = 5
Request when
elevator direction
is down then
not empty

elevator
change
direction

Let say, elevator reaches till 4th floor, the request is served

Request destination floor: 6
elevator current floor: 4

> current floor = It will be served when going up

elevator direction: DOWN
(as Nonpriority Queue is not empty)

High Priority Queue

6

Nonpriority Queue

2

→

Request destination floor: 5
elevator current floor: 4

> current floor = It will be served when going up.

elevator direction: DOWN (as High Priority is not empty)

High Priority

5, 6

Nonpriority

2

→

Let say, elevator reaches till 2nd floor request is served

elevator current floor: 2

changes direction now:

elevator direction: UP

High Priority

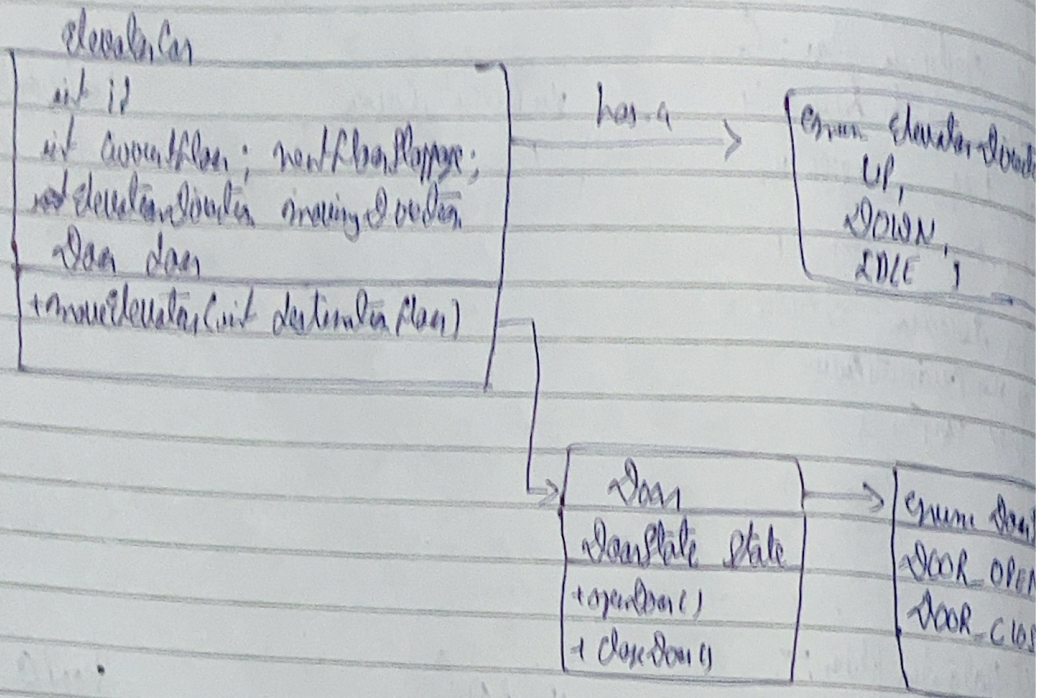
5, 6

→

Nonpriority

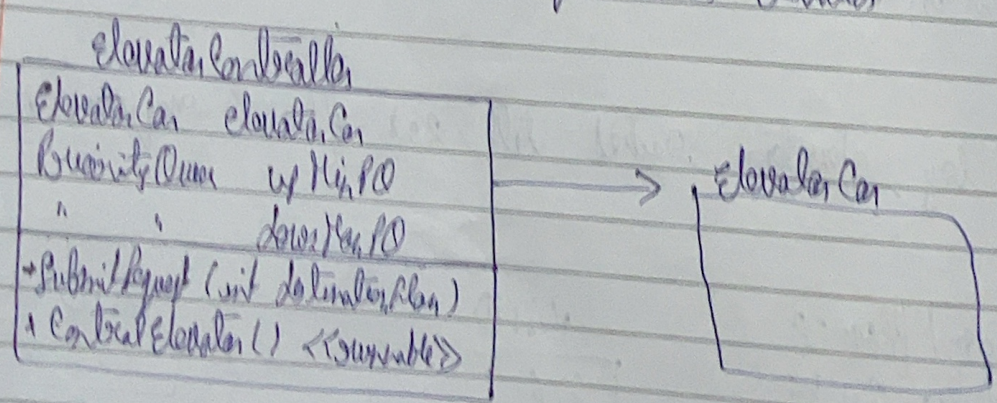
1. Elevator Car

An object, which moves to particular destination when asked to.

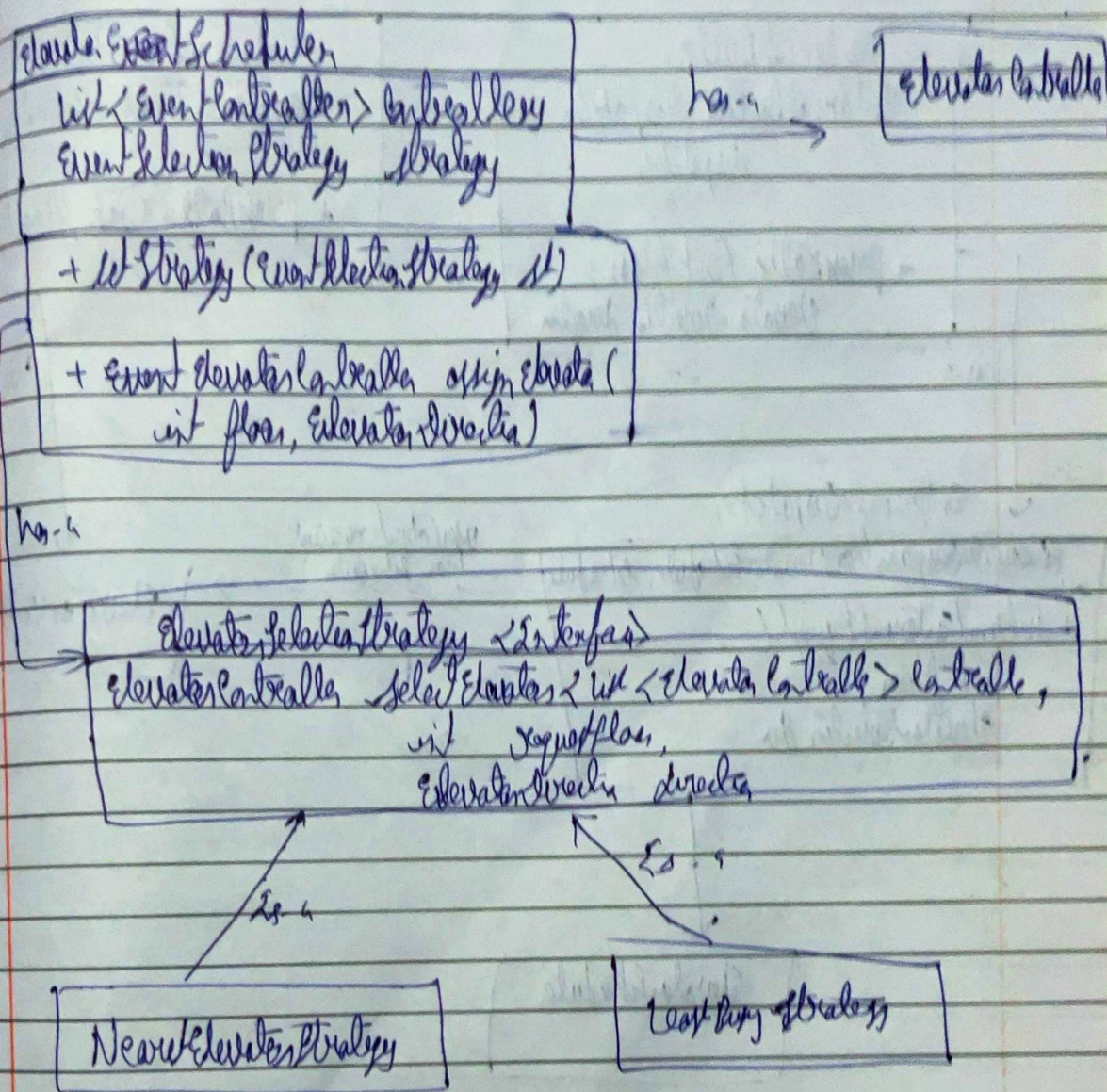


2. Elevator Controller

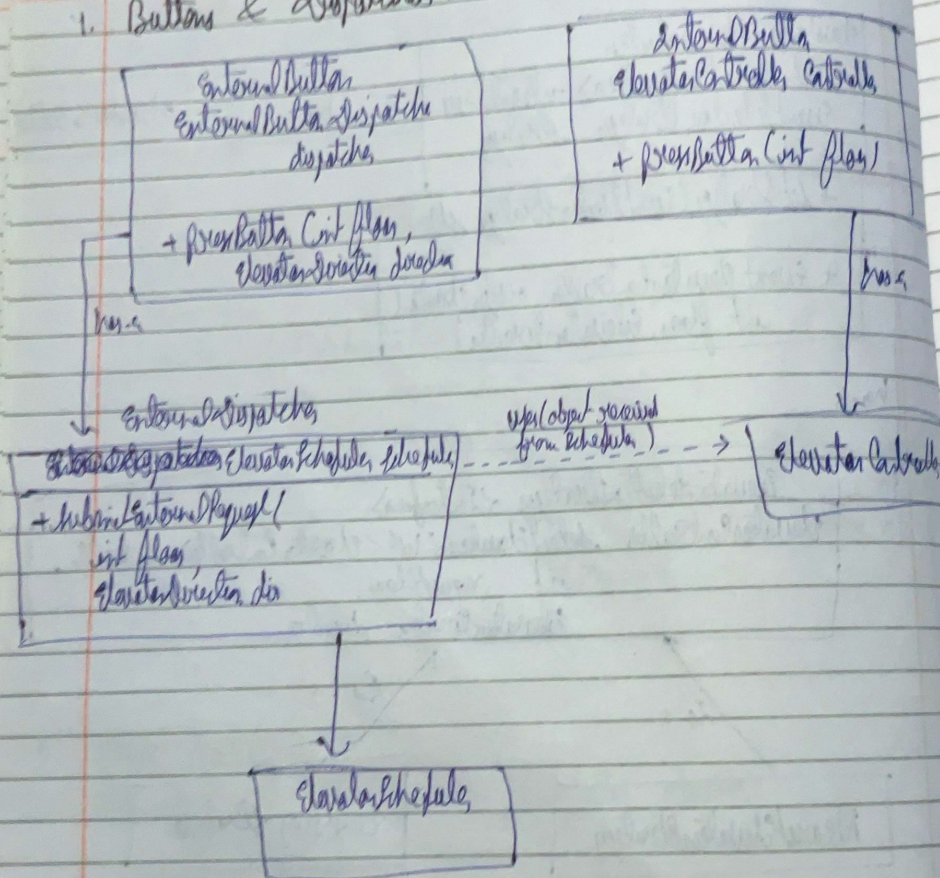
An object which controls a particular elevator



Geometrischer Scheduler:



1. Buttons & Dispatchers:



5. Plans &

Print
Link flow

5 Floors & Building

